23 — SQLite Vertiefung

JOIN, GROUP BY & Aggregate – Auswertungen direkt in SQL

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. `WHERE` & `ORDER BY` – gezielt filtern und sortieren
2. `JOIN` – Tabellen verbinden (und was dabei wirklich passiert)
3. `UPDATE` & `DELETE` – ändern und löschen (vorsichtig!)
4. Aggregatfunktionen – `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`
5. `GROUP BY` & `HAVING` – Auswertungen je Gruppe
6. Der Bauplan einer Abfrage – die logische Reihenfolge
7. **Klassiker: Top-Seller-Report** – live programmiert

Lernziel: Auswertungen dorthin verlagern, wo die Daten liegen – und **verstehen**, was ein `JOIN` und ein `GROUP BY` mechanisch tun.

Heute: nach jedem Konzept *Live-Demo* (ich, Veggie-Soles-DB) und *Sofort ausprobieren* (Sie, Notebook 23 – Chinook-Musikshop).

„Welches Produkt macht den meisten Umsatz?"

Ohne die Konzepte von heute

```
zeilen = cur.execute("SELECT ...").fetchall()
# ALLE Zeilen nach Python holen, dann:
# Schleifen, Dictionaries, sortieren ... Handarbeit
```

Mit den Konzepten von heute

```
SELECT produkt.name, SUM(preis * menge) AS umsatz
FROM bestellung JOIN produkt ON ...
GROUP BY produkt.name ORDER BY umsatz DESC;
```

Das DBMS beantwortet die Chef-Frage in **einer** Abfrage. Genau diese bauen wir heute Schritt für Schritt auf – bis zum Klassiker-Finale.

WHERE – gezielt filtern (Vertiefung)

```
SELECT name, preis FROM produkt
WHERE kategorie = 'Low-Cut' AND preis < 100;
```

Werkzeug	Beispiel
Vergleiche / Logik	`= != < >` · `AND` / `OR` (Klammern!)
Bereich (inklusiv)	`preis BETWEEN 90 AND 120`
Liste / Textmuster	`kategorie IN ('Boot','High-Cut')` · `name LIKE '%er%'`
fehlender Wert	`kategorie IS NULL` – **nicht** `= NULL`!

`WHERE` prüft die Bedingung für **jede Zeile einzeln** – wahr → Zeile bleibt. Ein `=`-Vergleich mit `NULL` ist **nie** wahr, darum `IS NULL`.

ORDER BY – sortieren

```sql
-- guenstigste zuerst:
SELECT name, preis FROM produkt
ORDER BY preis ASC;
-- nach Kategorie, dann Preis absteigend:
SELECT kategorie, name, preis FROM produkt
ORDER BY kategorie, preis DESC;
```

- ASC = aufsteigend (Standard, kann weg) · DESC = absteigend
- mehrere Spalten: die **erste** hat Vorrang, die zweite entscheidet bei Gleichstand
- ohne ORDER BY ist die Reihenfolge **nicht garantiert** – wer eine Ordnung braucht, muss sie bestellen

WHERE & ORDER BY – Demo & Übung

▶ **Live-Demo** – `01_where_und_order_by.py`: `BETWEEN`, `IN`, `LIKE`, `IS NULL`, mehrspaltiges Sortieren auf der Veggie-Soles-Produkttabelle

✎ **Sofort ausprobieren** – Notebook 23, Übungen 1.1, 1.2 & 2.1: Tracks über 5 Minuten; Rock-/Metal-Tracks für 0.99 (Klammern!); die 10 günstigsten Tracks

JOIN – Tabellen verbinden

```
SELECT kunde.name, bestellung.betrag
FROM bestellung
INNER JOIN kunde
  ON bestellung.kunde_id = kunde.kunde_id;
```

- Foliensatz 21 hat die Daten **zerlegt** (Redundanz raus) – JOIN setzt sie für die Abfrage wieder zusammen
- ON sagt, welche Zeilen zusammengehören: **Fremdschlüssel = Primärschlüssel**
- jede Ergebniszeile hat Spalten **beider** Tabellen – bei gleichen Namen Präfix `tabelle.spalte`

Unter der Haube: was ein `JOIN` wirklich tut

Gedankenmodell: **Kreuzprodukt** (alle Paare) $\rightarrow$ **Filter** (`ON`). 3 `produkt`-Zeilen $\times$ 2 `bestellung`-Zeilen = 6 Kombinationen:

`p.produkt_id`	`b.produkt_id`	`ON` erfüllt?
1	1	**wahr** ✓
1	3	falsch
2	1 · 3	falsch · falsch
3	1	falsch
3	3	**wahr** ✓

Foliensatz 21: eine Beziehung ist eine **Teilmenge des Kreuzprodukts** – genau das rechnet der `JOIN` aus: $\{(p, b) \in P \times B \mid p.\text{id} = b.\text{produkt_id}\}$. Deshalb ist der **Fremdschlüssel** die natürliche `ON`-Bedingung. (Das DBMS kürzt intern ab – das Ergebnis ist dasselbe.)

INNER vs. LEFT – als Mengen

JOIN als Mengenoperation: welche Zeilen landen im Ergebnis?

INNER JOIN: Treffer in beiden




```
FROM kunde JOIN bestellung ON ...
```

LEFT JOIN: alle links

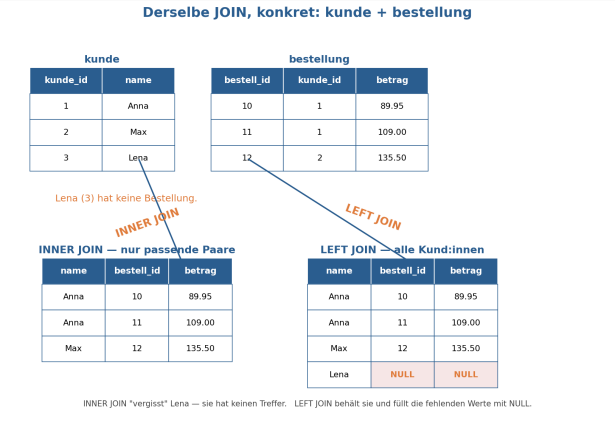



```
FROM kunde LEFT JOIN bestellung ON ...
```

Orange = diese Zeilen stehen im Ergebnis · bei LEFT JOIN bekommen Kund:innen ohne Bestellung NULL in den bestellung-Spalten.

- `INNER JOIN`: nur Treffer in **beiden** Tabellen · `LEFT JOIN`: **alle** Zeilen links, fehlt der Treffer rechts → `NULL`

INNER vs. LEFT – konkret



- `LEFT JOIN` + `WHERE bestellung.bestell_id IS NULL` findet gezielt „die **ohne**" – z. B. Kund:innen, die noch nie bestellt haben

JOIN – Demo & Übung

▶ **Live-Demo** – `02_inner_join.py` (bis zum 3-Tabellen-JOIN über `bestell-position`) · `03_left_join.py` (Kund:innen ohne Bestellung)

✎ **Sofort ausprobieren** – Notebook 23, Übungen 3.1 & 3.2: Tracks mit Genrenamen (`INNER JOIN`); Kunden ohne Rechnung (`LEFT JOIN` + `IS NULL`)

UPDATE & DELETE – ändern und löschen

```
UPDATE produkt SET preis = 79.90
WHERE name = 'Kork-Slip';
DELETE FROM produkt WHERE aktiv = 0;
```

- **immer mit `WHERE`!** Ohne `WHERE` trifft es *alle* Zeilen – und ist nicht rückgängig zu machen
- Best Practice: erst `SELECT` mit derselben `WHERE`-Bedingung – prüfen, *was* getroffen würde

▶ **Live-Demo** – `04_update_delete.py` (inkl. der auskommentierten „ohne WHERE"-Katastrophe)

✎ **Sofort ausprobieren** – Notebook 23, Übung 4.1: Künstler anlegen und per `UPDATE` umbenennen

Aggregatfunktionen – aus vielen Zeilen ein Wert

```
SELECT COUNT(*), AVG(preis), MAX(preis)
FROM produkt;
```

Funktion	liefert
`COUNT(*)` · `COUNT(spalte)`	Anzahl Zeilen · Anzahl Werte
`SUM(x)` · `AVG(x)`	Summe · Durchschnitt
`MIN(x)` · `MAX(x)`	kleinster · größter Wert

- ohne Gruppierung: das Ergebnis ist **eine einzige Zeile** – die ganze Tabelle ist eine Gruppe
- mathematisch: eine Funktion *Zeilenmenge* $\rightarrow$ *Zahl*, z. B. $\mathrm{SUM}(x) = \sum_{z \in \text{Zeilen}} x_z$

Unter der Haube: NULL in Aggregaten

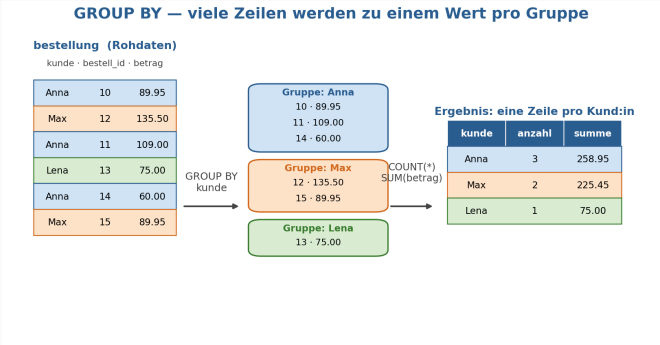
```sql
SELECT COUNT(*), COUNT(kategorie) FROM produkt;
-- Ergebnis: (6, 5)
```

Ausdruck	zählt	hier
`COUNT(*)`	alle **Zeilen**	6
`COUNT(kategorie)`	nur **Werte** ($\neq$ `NULL`)	5 (Pilz-Pump fehlt)

- alle Aggregate **überspringen** `NULL`: `AVG(kategorie-lose Spalte)` teilt nur durch die Anzahl vorhandener Werte
- Chinook: 3503 Tracks, aber `COUNT(Composer)` = 2525 – fast 1000 ohne Eintrag

`NULL` heißt „kein Wert" (Foliensatz 21) – was kein Wert ist, wird auch nicht mitgezählt oder mitgemittelt.

GROUP BY – Auswertung je Gruppe



- `GROUP BY kunde`: gleicher Wert = eine **Gruppe**, das Aggregat wird **pro Gruppe** berechnet

▶ **Live-Demo** – `05_aggregat_group_by.py`

Unter der Haube: `GROUP BY` mechanisch

```
SELECT kunde, SUM(betrag) FROM bestellung
GROUP BY kunde;
```

5 Bestellzeilen → partitionieren → 2 Gruppen → 2 Summen:

Gruppe	Beträge in der Gruppe	`SUM(betrag)`
Anna	89.95 · 109.00 · 60.00	258.95
Max	135.50 · 89.95	225.45

Die Falle: im SELECT darf neben Aggregaten nur stehen, **wonach gruppiert wurde**. Warum: die Gruppe *Anna* enthält **drei** verschiedene `bestell_id`-Werte – welchen davon sollte das DBMS in die eine Ergebniszeile schreiben?

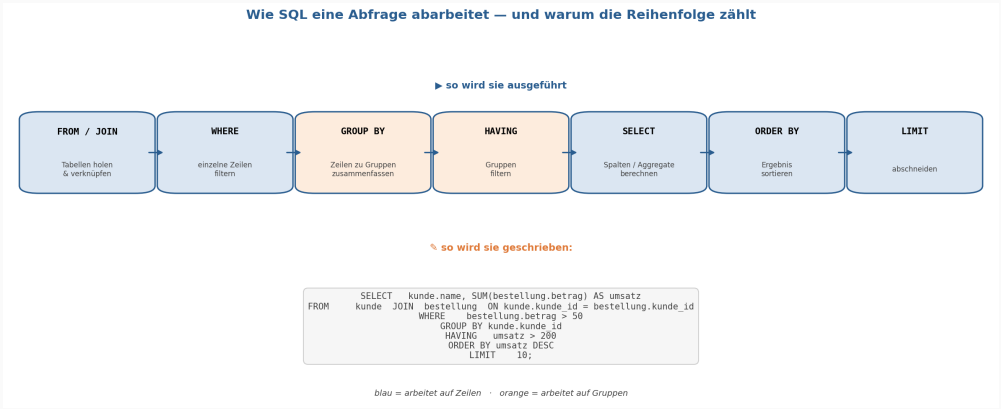
HAVING – Gruppen filtern

```sql
SELECT kunde, SUM(betrag) AS umsatz
FROM bestellung
GROUP BY kunde
HAVING umsatz > 230;     -- nur Anna (258.95)
```

Klausel	filtert	wirkt
`WHERE`	einzelne **Zeilen**	**vor** der Gruppierung
`HAVING`	ganze **Gruppen**	**nach** der Aggregation

✎ **Sofort ausprobieren** – Notebook 23, Übungen 5.1 & 5.2: Gesamtumsatz pro Kunde (`JOIN` + `GROUP BY`); dann nur Kunden über 40 (`HAVING`)

Unter der Haube: der Bauplan einer Abfrage



Geschrieben wird `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY` – *abgearbeitet* wird `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY`. **Deshalb** kennt `WHERE` keine Aggregate: beim Zeilenfiltern ist noch nichts summiert – Gruppenfragen gehören in `HAVING`.

Klassiker: Top-Seller-Report – die Aufgabe

Die Chef-Frage vom Anfang, jetzt lösbar:

- `produkt` (Name, Preis) · `bestellung` (Kunde, Produkt, Menge)
- je Produkt: Summe `preis * menge` + Stückzahl
- absteigend – Zeile 1 = **Top-Seller**: die **Menge** entscheidet!

So soll es laufen

```
Top-Seller-Report Veggie Soles
------------------------------
Eco-Sneaker   5 Stk   449.75 EUR
Hemp-High     3 Stk   327.00 EUR
Bambus-Boot   2 Stk   271.00 EUR

Top-Seller: Eco-Sneaker (449.75 EUR)
```

Klassiker: Top-Seller-Report – Live-Coding

▶ **Live-Coding ab leerem Editor** – Referenz: `90_klassiker_topseller.py`
Teilziele:

1. In-Memory-DB anlegen und füllen (`":memory:"` – Handwerk aus Foliensatz 22)
2. `JOIN` bauen, mit `SELECT *` prüfen: jede Bestellzeile kennt jetzt ihren Preis
3. Umsatz je Zeile: `preis * menge` – **vor** dem Gruppieren
4. `GROUP BY` Produkt + `SUM(...) AS umsatz` (Stückzahl: `SUM(menge)`)
5. `ORDER BY umsatz DESC` → Zeile 1 nehmen, formatiert ausgeben

✎ **Zum Selberlösen** – Handout *"Klassiker 23: Top-Seller-Report"* (`Klassiker/23_Klassiker_Topseller.pdf`). Vertiefung: Notebook 23, Abschlussübungen 3 & 4 (Chinook).

Cheat Card

Aufgabe	SQL
filtern	`WHERE ...` – AND/OR, BETWEEN, IN, LIKE, IS NULL
sortieren	`ORDER BY spalte [ASC|DESC]`
verbinden	`JOIN andere ON a.fk = andere.pk` (INNER / LEFT)
ändern / löschen	`UPDATE t SET s = w WHERE ...` · `DELETE FROM t WHERE ...`
zusammenfassen	`SELECT g, COUNT(*), SUM(x) FROM t GROUP BY g`
Gruppen filtern	`... GROUP BY g HAVING aggregat > wert`
Klassiker heute	`JOIN` → `preis*menge` → `GROUP BY` + `SUM` → `ORDER BY ... DESC`

Faustregel: `UPDATE`/`DELETE` *nie* ohne `WHERE`. `WHERE` filtert Zeilen (**vor** dem Gruppieren), `HAVING` filtert Gruppen (**danach**).

Ausblick: die Datenbank-Phase ist komplett

- **20** – warum Datenbanken (und nicht Excel), Anomalien
- **21 / 21b** – relationales Modell, Schlüssel, ER-Diagramme
- **22** – SQLite mit Python: anlegen, einfügen, abfragen
- **23** – verbinden, gruppieren, auswerten: der Report

Und der Kreis schließt sich: unser Top-Seller-Report ist **Text**. `fetchall()` liefert aber ganz normale Python-Listen – daraus machen die Werkzeuge aus Phase 4 (**Pandas, Matplotlib**) ein Diagramm fürs Management.

Vom ER-Entwurf über SQL bis zur Grafik: Sie können jetzt eine kleine Anwendung mit **dauerhafter, strukturierter, auswertbarer** Datenhaltung bauen.

Heute geübt

✓ `WHERE` mit `AND`/`OR`, `BETWEEN`, `IN`, `LIKE`, `IS NULL` · `ORDER BY`
✓ `JOIN` = Kreuzprodukt + `ON`-Filter – die Mathe aus Foliensatz 21 arbeitet
✓ `INNER` vs. `LEFT` – inkl. „die ohne Treffer" finden
✓ `UPDATE` / `DELETE` – immer mit `WHERE`
✓ Aggregate (`NULL` wird übersprungen!), `GROUP BY`, `HAVING`
✓ der Bauplan: `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY`
✓ **Klassiker Top-Seller-Report** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Top-Seller-Report** eigenständig lösen (Handout)
- „Sofort ausprobieren"-Übungen 1.1–5.2 und Abschlussübungen 1–4 im Notebook 23